Python: Your Magic Wand for AI

Week 1 – Lecture 1: Environment, Variables & Data Types

Course: Programming for AI
Dr. Prateek, AP
School of Artificial Intelligence

MTech AI

Today's Roadmap (60 minutes)

1. Welcome to the World of Python *10 min*
2. The Two Faces of Python *15 min*
3. Core Building Blocks: Data Types & Variables *20 min*
4. Rules of the Road: Naming & Expressions *15 min*

By the end of this lecture, you will be able to:

- Explain what Python is and why it is used in AI.
- Distinguish between the interactive shell and script mode.
- Identify `int`, `float`, and `str` data types.
- Create valid variable names and write simple expressions.

1. What is Python?

Definition

Python is a **high-level programming language** and an **interpreter** – a piece of software that reads your code, line by line, and carries out the instructions.

- You write instructions in a form close to English.
- The Python interpreter translates and executes them immediately.
- Created by Guido van Rossum; first released in 1991.
- Free, open-source, and runs on Windows, macOS, and Linux.

From Code to Output


```
print("Hello, AI world!")
```

Output: `Hello, AI world!`

Why Python for AI?

- **Readability:** Code looks close to plain English, so it is easier to write, debug, and share.
- **Productivity:** Fewer lines of code are needed compared to languages like C++ or Java for the same task.
- **Rich Ecosystem:** Libraries such as `NumPy`, `Pandas`, `TensorFlow`, and `PyTorch` are built for data science and AI.
- **Community Support:** A huge global community means abundant tutorials, forums, and documentation.

Key Idea

Python is optimized for *programmer productivity* and *code readability* – not necessarily for raw execution speed.

2. The Two Faces of Python

Python code can be run in two main ways:

1. **Interactive Shell (REPL)**
2. **File Editor (Script Mode)**

REPL stands for *Read–Evaluate–Print–Loop*: the interpreter *reads* what you type, *evaluates* it, *prints* the result, and *loops* back for the next line.

The Interactive Shell (REPL)

Analogy

Like a *"wizard-in-training" session*: you type one line, and you get an instant result. Perfect for quick experiments.

```
>>> 5 + 3
8
>>> "Hello" + " World"
'Hello World'
>>> 10 / 4
2.5
```

When to use it:

- Testing a small idea quickly.
- Checking how an operator or function behaves.
- Nothing you type here is saved automatically.

The File Editor (Script Mode)

Analogy

Where you write entire *"scrolls"* (programs) – saved to a file with a `.py` extension and run whenever needed.

```
# saved as: greet.py
name = "Aditi"
print("Welcome to Python,", name)
print("Let's build AI together!")
```

When to use it:

- Writing programs longer than a line or two.
- Code that must be reused, shared, or submitted.
- Building complete AI/ML projects.

REPL vs. Script Mode

Aspect	**Interactive Shell**	**File Editor**
Purpose	Quick experiments	Complete programs
Saved?	No	Yes (`.py` file)
Execution	Line by line	Whole file at once
Best for	Learning, testing	Real projects

3. Core Building Blocks: Data Types

Idea

Every value in Python belongs to a **data type** – a category that tells Python what kind of value it is and what operations are allowed on it.

Today, we focus on three fundamental types:

- `int` – Integers
- `float` – Floating-point numbers
- `str` – Strings (text)

Integers (`int`)

Definition: Whole numbers, positive or negative, with no decimal point.

```
>>> age = 17
>>> type(age)
<class 'int'>
>>> temperature = -5
>>> big_number = 1000000
```

- Examples: 42, 0, -17, 1000000
- Used for counting, indexing, and whole-number quantities.

Floating-Point Numbers (`float`)

Definition: Numbers that contain a decimal point.

```
>>> pi_value = 3.14
>>> type(pi_value)
<class 'float'>
>>> height = 5.6
>>> gpa = 9.2
```

- Examples: `3.14`, `0.5`, `-2.75`
- Used whenever precision beyond whole numbers is needed – for example, measurements or probabilities in AI models.

Strings (str)

Definition: Text data, always enclosed in quotes – single ('...') or double ("...").

```
>>> name = "Ravi"
>>> type(name)
<class 'str'>
>>> greeting = 'Good Morning'
>>> student_id = "12A-07"
```

- Even digits inside quotes are text, not numbers: "123" is a str, not an int.
- Strings can be joined using + (concatenation).

Checking a Data Type: `type()`

Python provides the built-in `type()` function to check the data type of any value.

```
>>> type(10)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type("AI")
<class 'str'>
```

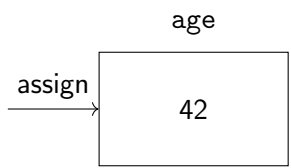
Try It Yourself

What is the type of `"10"`? What about `10.0`?

Variables: Labeled Boxes in Memory

Analogy

A variable is like a **labeled box** in the computer's memory where a value is stored, so it can be used or changed later.




```
age = 17
```

Here, age is the *label*, = is the **assignment operator**, and 17 is the value placed inside the box.

Working with Variables

```
student_name = "Meera"
student_age = 17
average_score = 88.5

print(student_name)       # Meera
print(student_age)        # 17

student_age = 18          # value can change!
print(student_age)        # 18
```

- A variable's value **can be updated** at any time.
- Python figures out the data type automatically – this is called **dynamic typing**.

4. Rules of the Road: Naming Variables

Valid variable names must follow these rules:

- Must be a **single word** (no spaces).
- Can use only **letters, digits, and underscores** (_).
- **Cannot start with a digit.**
- Cannot be a Python **keyword** (e.g., `if`, `for`, `print`).
- Names are **case-sensitive**: `Age` and `age` are different variables.

Valid vs. Invalid Names

Valid

```
student_name
age2
_score
totalMarks
class_XII
```

Invalid

```
2nd_score       # starts with
    digit
student name    # contains space
class           # a keyword
total-marks     # hyphen not
    allowed
```

Good Practice

Choose descriptive names: `student_score` is clearer than `s`.

Expressions

Definition

An **expression** combines values and operators, and it always **evaluates down to a single value**.

```
>>> 4 + 5
9
>>> 2 * 3.5
7.0
>>> 10 - 4 * 2
2
>>> "AI" + " " + "Class"
'AI Class'
```

Common Arithmetic Operators

Operator	Meaning	Example
+	Addition	`5 + 2` $\rightarrow 7$
-	Subtraction	`5 - 2` $\rightarrow 3$
*	Multiplication	`5 * 2` $\rightarrow 10$
/	Division	`5 / 2` $\rightarrow$ `2.5`
//	Floor Division	`5 // 2` $\rightarrow 2$
%	Modulus (remainder)	`5 % 2` $\rightarrow 1$
**	Exponent (power)	`5 ** 2` $\rightarrow 25$

Expressions with Variables

```
length = 5
breadth = 3
area = length * breadth
print(area)           # 15

price = 49.5
quantity = 3
total_cost = price * quantity
print(total_cost)     # 148.5
```

Notice: an expression using a `float` and an `int` together produces a `float` result.

Summary: What We Learned Today

- Python is an **interpreted**, readable, productive language – ideal for AI development.
- Two ways to run code: the **REPL** (quick tests) and the **File Editor** (saved programs).
- Three core data types: `int`, `float`, and `str`.
- **Variables** are labeled boxes storing values; names must follow Python's naming rules.
- **Expressions** combine values and operators to produce a single result.

Practice Before Next Class

1. Open the Python shell and try five different arithmetic expressions.
2. Create variables for your `name`, `age`, and `favorite_subject`.
3. Use `type()` to check the data type of each variable.
4. Identify which of the following names are valid: `1data`, `my_data`, `my data`, `data_1`.

Next Lecture: Input/Output, Type Conversion, and Basic Operators in Depth.

Questions? Thank you!